

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC - KỸ THUẬT MÁY TÍNH



Đồ án thiết kế luận lý

Sử dụng ngôn ngữ Verilog để hiện thực

Bộ xử lý MIPS đơn chu kì.

GVHD: PGS.TS Phạm Quốc Cường
SV: Ngô Đức Anh - 2210077
Trần Như Mai Anh - 2210140
Hồ Quốc Đạt - 2210671

TP. HỒ CHÍ MINH, THÁNG 1/2025

Mục lục

1	Lý thuyết:	3
1.1	Lý thuyết kiến trúc tập lệnh	3
1.2	Lý thuyết Kiến trúc MIPS	3
1.2.1	Khối đọc lệnh	4
1.2.2	Khối thanh ghi	5
1.2.3	Khối ALU	5
1.2.4	Khối điều khiển ALU	6
1.2.5	Khối bộ nhớ dữ liệu	7
1.2.6	State diagram	8
1.3	Lý thuyết ngôn ngữ Verilog	9
1.3.1	Giới thiệu	9
1.3.2	Structural Model	9
1.3.3	RTL Model	10
1.3.4	Behavioral Model	12
2	Hiện thực:	18
2.1	Module style	18
2.2	Input và output của bộ xử lý	18
2.3	Module hỗ trợ	18
2.4	Module register_file	18
2.5	Module alu	19
2.6	Module alu_control	20
2.7	Module control_unit	21
2.8	Module data_memory	21
2.9	Module instruction_memory	22
2.10	Schematic của bộ xử lý	22
3	Mô phỏng	24
3.1	Source code dùng để simulation	24
3.2	Behavior simulation	24
3.2.1	Waveform of submodule	24
3.2.2	Top module	26
3.3	Post-Synthesis Simulation-Functional	28
4	Ước lượng	29
4.1	Ước lượng tài nguyên	29
4.2	Tần số hoạt động	29
4.3	Ước lượng công suất	30
5	Mở rộng	31
5.1	Giới thiệu	31
5.2	Các kiến trúc được mở rộng để phục vụ yêu cầu mở rộng tập lệnh	31
5.2.1	Khối data memory	31
5.2.2	Khối control	31
5.2.3	Khối ALU	32
5.3	Simulation	33
5.3.1	Chương trình tính số Fibonacci	33
5.3.2	Một số testcase khác để kiểm tra full tập lệnh	36



5.4	Ước lượng tài nguyên sau cải tiến	38
5.4.1	Thống kê tài nguyên	38
5.4.2	Ước lượng tần số	39
5.4.3	Ước lượng hiệu năng	39
6	Code	39

1 Lý thuyết:

1.1 Lý thuyết kiến trúc tập lệnh

Trong hệ thống, hợp ngữ đóng vai trò trung gian kết hợp giữa phần mềm và phần cứng. Để máy tính có thể thực hiện được các chương trình phần mềm, trình biên dịch sẽ tiến hành biên dịch chương trình từ các ngôn ngữ cấp cao (GO, Python, C++, ...) về các kiến trúc tập lệnh (ISA). Sau đó máy tính sẽ thực thi chương trình dựa trên các lệnh của ISA đã biên dịch. Một số ISA nổi bật có thể kể đến như:

- MIPS
- ARM (Phổ biến với các thiết bị)
- x86 (Phổ biến với các thiết bị máy tính bàn và laptop)

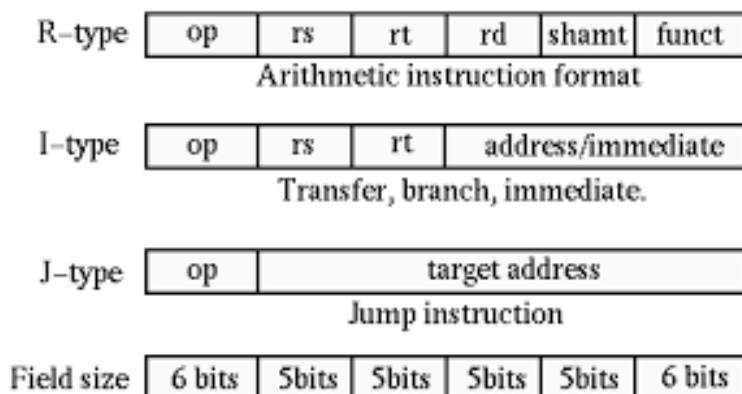
Các kiến trúc tập lệnh hỗ trợ 3 loại lệnh chính bao gồm:

- Nhóm 1: Tập các lệnh luận lý/logic thực hiện các phép toán và xử lý trên bit ví dụ add, dịch bit, and, or, ...
- Nhóm 2: Tập các lệnh chuyển dữ liệu chịu trách nhiệm mang dữ liệu giữa thanh ghi với bộ nhớ.
- Nhóm 3: Tập các lệnh điều khiển luồng chương trình như jump, branch, ...

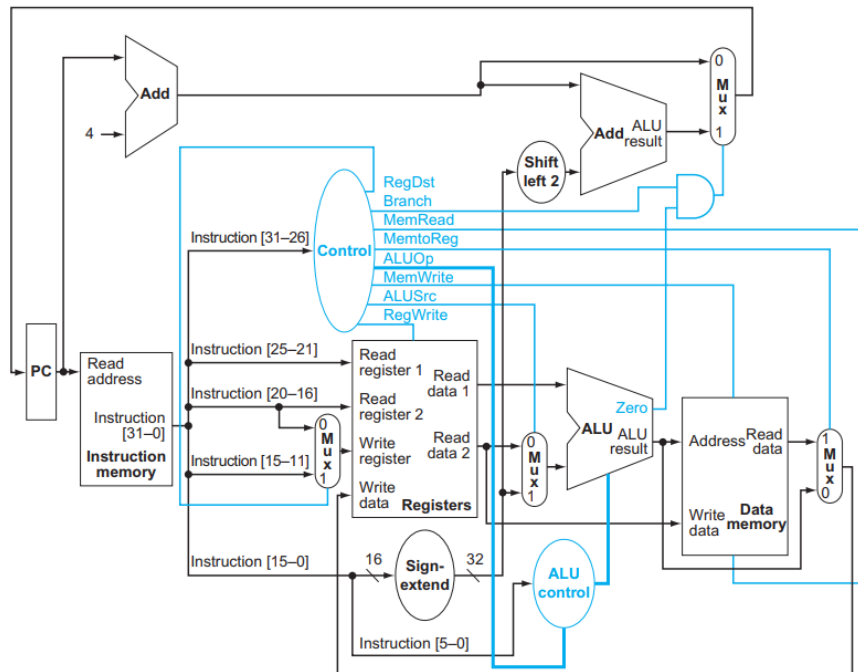
1.2 Lý thuyết Kiến trúc MIPS

MIPS là kiến trúc tập lệnh RISC được phát triển bởi John L. Hennessy. Là kiến trúc tập lệnh RISC, tập lệnh MIPS có độ dài cố định là 32 bits, bao gồm 32 thanh ghi chung 32 bits (GPIO), 32 thanh ghi số thực và một số thanh ghi đặc biệt phục vụ cho việc thực thi của kiến trúc. Vì chiều dài câu lệnh và thanh ghi là cố định, do đó kiến trúc mips cung cấp hiệu năng khá cao, dễ dàng tiếp cận cũng như bảo trì, tuy nhiên lại mang đến sự đánh đổi về tính đa dạng của tập lệnh.

Về tập lệnh cho kiến trúc, tập lệnh kiến trúc hiện chia thành 3 loại lệnh thanh ghi (R-type), lệnh hằng số (I-type) và lệnh nhảy (J-type) được quy định như sau

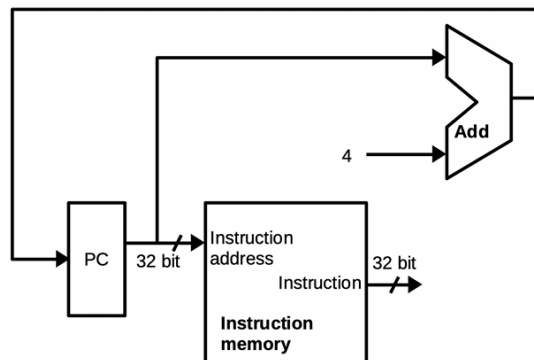


Hình 1: Quy ước tập lệnh



Hình 2: Góc nhìn ở mức cao của hiện thực kiến trúc MIPS

1.2.1 Khối đọc lệnh



Hình 3: Khối đọc lệnh

Để có thể thực thi chương trình lệnh cần phải được tải từ bộ nhớ khối IM sẽ nhận vào lệnh từ bộ nhớ dựa trên con trỏ chương trình (PC) được đưa vào và trả về nội dung tại địa chỉ PC có độ dài là 32 bits. Sau đó thực hiện tính toán địa chỉ câu lệnh tiếp theo bằng cách tăng PC lên 4 bytes. Cấu trúc khối tải lệnh được quy định như sau:

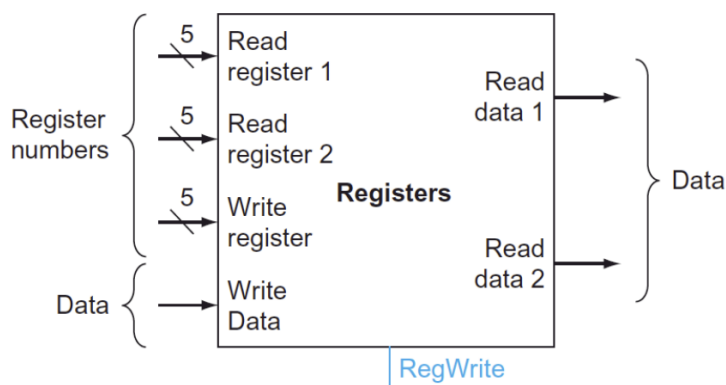
- Đầu vào

- Địa chỉ lệnh tiếp theo cần được tải lên: 32 bits
- Đầu ra
 - Dữ liệu của lệnh tiếp theo được tải lên từ bộ nhớ: 32 bits

1.2.2 Khối thanh ghi

Trong kiến trúc MIPS single-cycle, bộ thanh ghi (register file) đóng vai trò quan trọng trong việc lưu trữ và truy xuất dữ liệu tạm thời. Bộ thanh ghi MIPS thường chứa 32 thanh ghi đa dụng, 32 thanh ghi số thực, mỗi thanh ghi có kích thước 32-bit. Cấu trúc của khối thanh ghi gồm

- Đầu vào
 - Hai địa chỉ thanh ghi nguồn read register 1, read register 2: 5 bit
 - Một địa chỉ thanh ghi đích write register: 5 bit
 - Dữ liệu để ghi vào thanh ghi write data: 32 bit
 - Một tín hiệu điều khiển ghi RegWrite: 1 bit
- Đầu ra
 - Hai giá trị đọc từ các thanh ghi nguồn Read data 1 và Read data 2: 32 bit



ID & WB

1.2.3 Khối ALU

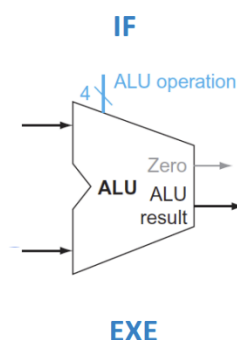
Trong kiến trúc MIPS single-cycle, Arithmetic Logic Unit (ALU) là thành phần thực hiện các phép toán số học và logic. Cơ chế hiện thực có thể cần được giải mã 2 lớp để xác định hành vi cụ thể của khối ALU.

- Lớp 1: 6 bits opcode ta thu được 2 bits ALUOp

- Lớp 2: Từ 2 bits ALUOp có thể kết hợp với 6 bits (function code) để xác định 4 bit ALU operation

Cấu trúc của bộ ALU gồm

- Đầu vào
 - Hai toán hạng operand 1 và operand 2: 32 bit
 - Tín hiệu điều khiển ALU Control để xác định phép toán: 4 bit
- Đầu ra
 - Kết quả của phép toán Result: 43 bit
 - Tín hiệu logic hỗ trợ lệnh nhánh Zero: 1 bit



1.2.4 Khối điều khiển ALU

Bộ điều khiển ALU (ALU Control Unit) trong kiến trúc MIPS single-cycle là thành phần xác định phép toán cụ thể mà ALU sẽ thực hiện.

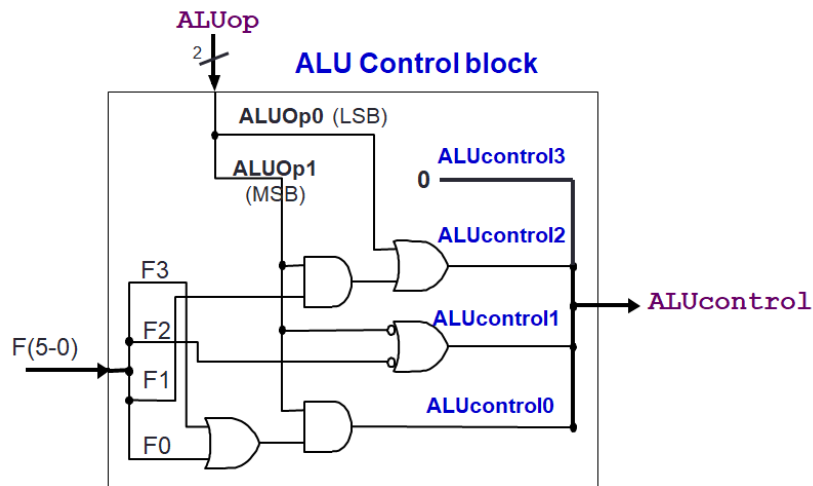
Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111

Hình 4: Bảng xác định ALU operation

Cấu trúc của bộ điều khiển ALU gồm

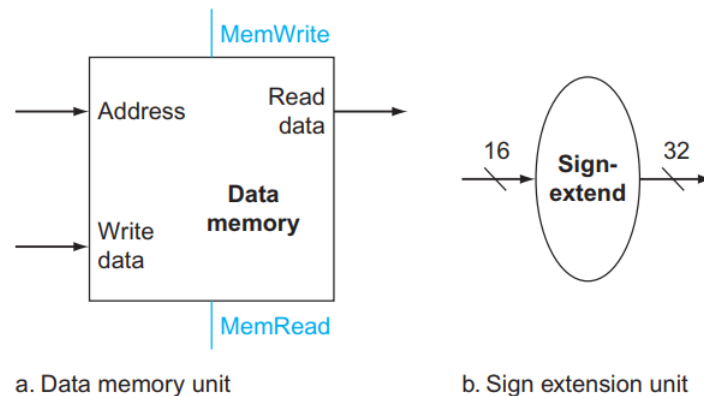
- Đầu vào
 - Tín hiệu từ bộ điều khiển chính ALUOp: 2 bit

- Trường 6 bit từ lệnh R-type, chứa thông tin về phép toán cụ thể
- Đầu ra
 - Tín hiệu điều khiển ALU Control để xác định phép toán : 4 bit



1.2.5 Khối bộ nhớ dữ liệu

Khối bộ nhớ dữ liệu có tác vụ thực hiện thao tác truy xuất nội dung bộ nhớ được sử dụng cho các câu lệnh giao tiếp với bộ nhớ như load, store. Cấu trúc của khối bộ nhớ dữ liệu gồm:

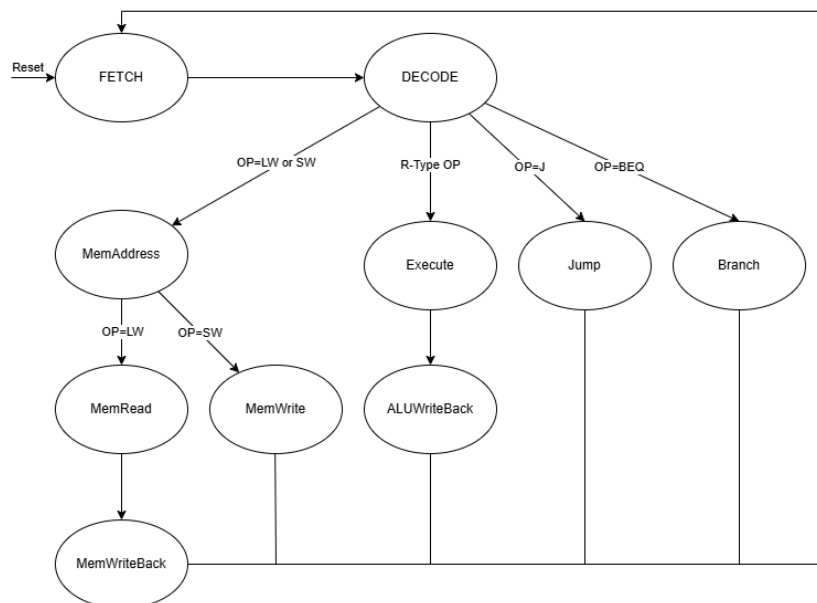


Hình 5: Khối bộ nhớ

- Đầu vào
 - Địa chỉ vùng nhớ cần đọc hay ghi: 32 bits
 - Dữ liệu được dùng để ghi vào vùng nhớ: 32 bits
 - Tín hiệu đọc bộ nhớ - MemRead: 1 bit

- Tín hiệu ghi bộ nhớ - MemWrite: 1 bit
- Đầu ra
 - Dữ liệu của vùng nhớ được đọc: 32 bit

1.2.6 State diagram



1.3 Lý thuyết ngôn ngữ Verilog

1.3.1 Giới thiệu

Verilog là một ngôn ngữ mô tả phần cứng (HDL) được sử dụng để mô tả các hệ thống và mạch số dưới dạng mã. Nó được phát triển bởi Gateway Design Automation vào giữa những năm 1980 và sau đó được Cadence Design Systems mua lại.

Verilog được sử dụng rộng rãi để thiết kế và xác minh các hệ thống kỹ thuật số và tín hiệu hỗn hợp, bao gồm cả mạch tích hợp chuyên dụng (ASIC) và mảng cổng lập trình trường (FPGA). Nó hỗ trợ một loạt các mức độ trừu tượng, từ cấu trúc đến hành vi, và được sử dụng cho cả thiết kế dựa trên mô phỏng và thiết kế dựa trên tổng hợp.

Vì tính linh hoạt của nó, ngôn ngữ này được sử dụng để mô tả mạch theo cơ chế phân lớp. Bắt đầu từ các cổng logic và flip-flop và xây dựng lên các khối chức năng và hệ thống phức tạp hơn. Ngoài ra, vì bản chất linh hoạt được chuyển thể từ việc lắp ráp mạch thật sang mô hình hóa nó bằng HDL nên chúng ta sẽ không phải tốn thời gian cho những mạch đã được thiết kế từ trước, mà có thể tái sử dụng chúng chỉ bằng những lệnh gọi primitives đó. Vì vậy làm cho thiết kế ngày càng đơn giản và ta không còn quá phải quan tâm về vấn đề lắp ráp mạch, mà chuyển sang tập trung xây dựng những kiến trúc phức tạp hơn.

Ngoài ra, ngôn ngữ HDL còn có 1 số công cụ hỗ trợ như Vivado, ModelSim,... Giúp chúng ta dễ dàng thiết kế, kiểm thử, đánh giá được hiệu năng để tối ưu hệ thống mà không cần phải thao tác trực tiếp lên mạch thật. Làm giảm đáng kể thời gian và chi phí trong quá trình hiện thực dự án.

Verilog cung cấp 3 độ mô hình hóa với mỗi loại có chức năng và công dụng khác nhau:

- **Structural Model:** Hay còn được gọi là mô hình cấu trúc. Mô hình này cho phép chúng ta thao tác ở mức độ trừu tượng thấp nhất. Làm việc trên các cổng luận lý cơ bản như: and, or, nor,... Mô hình này giúp chúng ta định nghĩa được các Primitives để sau này chúng ta sẽ gọi lại nó như các submodule trong module tổng thể của mình. Làm việc ở mức này giống việc chúng ta đang nối dây giữa các khối sẵn có lại với nhau.
- **RTL Model:** Ở mức này, chúng ta đã lên tới mức trừu tượng cao hơn, thay vì phải gọi các cổng luận lý cơ bản như mô hình cấu trúc, chúng ta sẽ làm việc với các toán tử, và toán hạng, thay cho các cổng cơ bản đó. Giúp ích cho chúng ta dễ dàng thao tác hơn, dễ debug, và thao tác được với nhiều biến hơn thông qua các biểu thức toán tử 1 ngôi, 2 ngôi và 3 ngôi.
- **Behavioral Model:** Mức này là mức trừu tượng cao nhất, nó chính là loại tạo nên sức mạnh cho ngôn ngữ Verilog này. Nó giúp chúng ta khai triển ý tưởng gần giống phần mềm, hiện thực theo kiểu thuật toán gần gũi với chúng ta hơn. Bên cạnh đó, nó còn cung cấp cho chúng ta các thao tác để tạo benchtest dễ dàng và nhanh hơn.

Dưới đây, chúng em xin phép cung cấp những kiến thức ở môn thiết kế luận lý, mà được dùng để làm trong đồ án này sẽ được ôn lại để hiện thực.

1.3.2 Structural Model

Trong mô hình này, kiến thức cần có đầu tiên là về Net(đường dẫn tín hiệu). Trong bài này, chúng em sử dụng chủ yếu là loại **wire**. **Wire** đại diện cho các kết nối của các thành phần khác nhau trong mạch. Chức năng của **wire**:

- Kết nối các phần tử: Wire đóng vai trò như những "cầu nối" kết nối các cổng logic, thanh ghi, và các module khác trong mạch.

- Truyền dữ liệu: Dữ liệu (bit 0 hoặc 1) được truyền đi trên các wire từ nguồn (output của một cổng logic) đến đích (input của một cổng logic khác).

Đặc điểm của **wire**:

- Không có giá trị mặc định: Trái ngược với các loại biến khác như register, wire không có giá trị mặc định. Giá trị của wire tại một thời điểm bất kỳ phụ thuộc hoàn toàn vào giá trị của các tín hiệu kết nối vào nó.
- Thay đổi theo thời gian: Giá trị trên wire có thể thay đổi liên tục theo thời gian khi các tín hiệu đầu vào thay đổi.
- Không có khả năng lưu trữ: Wire không có khả năng lưu trữ giá trị. Khi không có tín hiệu nào tác động lên wire, giá trị của nó sẽ không xác định.

Tiếp theo, là các loại primitives được định nghĩa bởi người dùng (User-Defined Primitives-UDP).

Định nghĩa: UDP là những primitives mà người thiết kế tự định nghĩa để mô tả các hành vi logic cụ thể. Chúng có thể đơn giản như một cổng logic đặc biệt hoặc phức tạp như một khối chức năng tùy biến. Ứng dụng của UDP trong bài tập lớn này:

- Mô hình hóa các thành phần không có sẵn trong thư viện: UDP cho phép chúng em mô hình hóa các thành phần tùy chỉnh hoặc các thành phần không có sẵn trong thư viện của công cụ tổng hợp.
- Tăng hiệu suất mô phỏng: Bằng cách sử dụng UDP, chúng em có thể mô hình hóa một phần của mạch ở mức độ trừu tượng cao hơn, giúp giảm thời gian mô phỏng.
- Bảo mật thiết kế: UDP có thể được sử dụng để bảo vệ các phần thiết kế quan trọng bằng cách ẩn đi các chi tiết thực hiện bên trong.

Với 2 kiến thức này của mô hình cấu trúc, chúng em xây dựng thành các khối chức năng riêng lẻ có bên trong bộ xử lý MIPS đơn chu kì. Và kết nối chúng bằng các wire vào ra model.

1.3.3 RTL Model

RTL là viết tắt của Register Transfer Level. Đây là một cấp độ trừu tượng trong thiết kế vi mạch, mô tả cách thức dữ liệu được truyền giữa các thanh ghi (register) thông qua các phép toán. Mô hình RTL cho phép ta thao tác giữa các thành phần bên trong mạch ở mức trừu tượng cao hơn thông qua các toán tử. Và toàn bộ code trong mô hình này luôn luôn tổng hợp được, và sinh ra mạch mà không sợ hiện tượng không thể tổng hợp.

Ưu điểm của việc sử dụng mô hình RTL

- Năng suất cao: Giảm thời gian thiết kế và tăng năng suất.
- Dễ dàng tái sử dụng: Các mô-đun RTL có thể được tái sử dụng trong các thiết kế khác.
- Dễ dàng kiểm tra và sửa lỗi: Các công cụ mô phỏng và kiểm tra có thể dễ dàng áp dụng cho mã RTL.

Với mô hình này, chúng em sử dụng dựa trên bản tổng hợp toán tử cơ bản sau:

Operator	Name	Group
[]	Select	
()	Bracket	
!	Negation (inverse)	Logical
~	Negation (not)	Bit-wise
&	Reduction AND	Reduction
	Reduction OR	
~&	Reduction NAND	
~	Reduction NOR	
^	Reduction XOR	
~^ or ^~	Reduction XNOR	
+	Positive (unary)	Arithmetic
-	Negative (unary)	
{ }	Concatenation	Concat.
{ }	Replication	Repl.
*	Multiplication	Arithmetic
/	Division	
%	Modulus	
+	Addition	
-	Subtraction	

Operator	Name	Group
<<	Shift left	Shift
>>	Shift right	
>	Greater	Relational
>=	Greater or equal	
<	Less	
<=	Less or equal	
==	Equal (logic)	Equal
!=	Not equal (logic)	
===	Equal (case)	
!==	Not equal (case)	
&	bit-wise AND	Bit-wise
^	bit-wise XOR	
^~ or ~^	bit-wise XNOR	
	bit-wise OR	
&&	logical AND	Logic
	logical OR	
?:	Conditional	Conditional

Bảng trên cung cấp với các thao tác trên các loại toán tử 1 ngôi, 2 ngôi, 3 ngôi,... dưới đây chúng em sẽ miêu tả các hoạt động của từng nhóm toán tử:

- Toán tử 1 ngôi:** Toán tử một ngôi trong Verilog là những toán tử chỉ tác động lên một toán hạng duy nhất. Chúng thường được sử dụng để thực hiện các phép toán đơn giản như phủ định, số đối, tăng giảm giá trị, thu gọn biểu thức,... chúng là các toán tử thuộc nhóm Reduction và Bit-wise như trên bảng cung cấp.
- Toán tử 2 ngôi:** Toán tử hai ngôi trong Verilog là những toán tử tác động lên hai toán hạng để tạo ra một kết quả mới. Chúng được sử dụng rộng rãi trong các phép tính số học, logic và so sánh. Bên trên chúng thuộc các nhóm Arithmetic, Equal, Shift, Relational và Logic.
- Toán tử 3 ngôi:** Toán tử 3 ngôi là loại toán tử đặc biệt nhất, chúng thuộc nhóm Conditional và chúng thực hiện so sánh tương tự như if, else.
- Nhóm toán tử mở rộng:** Toán tử mở rộng ở trên chính là các toán tử như Concat và Repl. Nó hoạt động như chính tên gọi của nó. Concat nó sẽ nối các toán hạng của chúng ta lại với nhau. Repl thì nó sẽ nhân bản giá trị bit của chúng ta lên theo số lần chúng ta khai báo ở trong biểu thức.

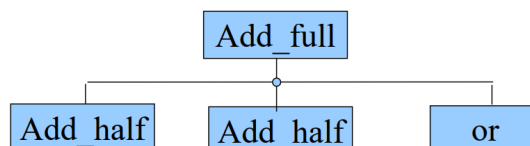
Chiều dài quy ước sau mỗi lần thực hiện phép toán:

Expression	Bit length	Comments
Unsize constant number	Same as integer (32 bit)	
Sized constant number	As given	
i <op> j, where <op> is: +, -, *, /, %, , ^, ~^	Max(L(i),L(j))	
i <op> j, where <op> is: ==, !=, ==, !=, &&, , >, >=, <, <=	1 bit	Operands are sized to Max(L(i),L(j))
i <op> j, where <op> is: &, ~&, , ~ , ^, ~^, !	1 bit	
i <op> j, where <op> is: >>, <<	L(i)	

Ngoài ra, thành phần không thể thiếu đó chính là Assign Syntax:

```
1 assign <LHS> = <RHS expression>;
```

Dưới đây là 1 ví dụ nhỏ để cho thấy sự tối ưu hơn khi sử dụng mô hình RTL so với mô hình cấu trúc:



```
module Add_half(sum, cout, a, b);
    output sum, cout;
    input a, b;
    assign sum = a ^ b;
    assign cout = a & b;
endmodule
```

```
module Add_full(c_out, sum, a, b, c_in);
    output sum, c_out;
    input a, b, c_in;
    wire psum, c1, c2;
    Add_half AH1(partsum, c1, a, b);
    Add_half AH2(sum, c2, psum, c_in);
    assign c_out = c1 | c2;
endmodule
```

1.3.4 Behavioral Model

Mô hình hành vi (Behavioral model) trong Verilog là một cách tiếp cận mô tả mạch điện tử ở cấp độ trừu tượng cao. Thay vì miêu tả chi tiết các cổng logic và kết nối giữa chúng như ở mức cổng (gate level), mô hình hành vi tập trung vào việc mô tả hành vi mong muốn của mạch bằng các câu lệnh mô tả cao cấp. Điều này giúp cho quá trình thiết kế mạch trở nên dễ dàng và nhanh chóng hơn, đặc biệt đối với các mạch phức tạp.

Đặc điểm chính:

- Trừu tượng cao nhất: Mô tả hành vi của mạch bằng các câu lệnh giống như ngôn ngữ lập trình, không cần quan tâm đến cấu trúc vật lý của mạch. Đặc tả thông qua hoàn toàn bằng thuật toán.
- Linh hoạt: Dễ dàng thay đổi và sửa đổi mô hình để đáp ứng các yêu cầu thiết kế khác nhau.
- Hiệu quả: Mô hình hành vi thường ngắn gọn và dễ đọc hơn so với mô hình mức cổng.
- Phù hợp với các công cụ tổng hợp: Các công cụ tổng hợp hiện đại có thể tự động chuyển đổi mô hình hành vi thành các mạch logic thực tế.
- Giảm thời gian, công sức rất nhiều lần.

Bên cạnh đó, cũng có nhiều nhược điểm lớn như:

- Không phải lúc nào cũng có thể tổng hợp được mạch, trường hợp xấu nhất, với những thuật toán dùng để miêu tả mạch, có thể sẽ không được tổng hợp và dẫn đến bị bỏ sót chức năng mà không hay biết.
- Phải quan tâm kĩ hơn về cách thức đặc tả, nếu không quan tâm kĩ, rất dễ xảy ra các hiện tượng không mong muốn như latch, glitch, hazard.

Trên mô hình hành vi này, có 4 cấu trúc cơ bản trong mô hình hành vi:

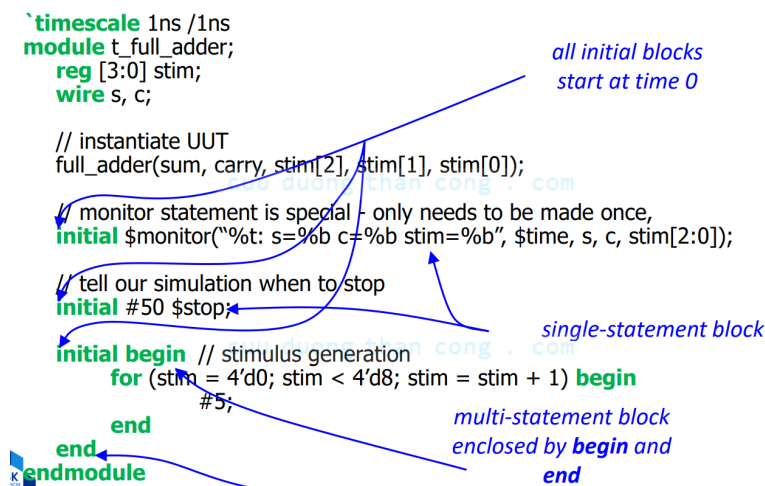
- Cấu trúc Initial
- Cấu trúc Always
- Cấu trúc Function
- Cấu trúc Task

Trong phần ứng dụng này chúng em sử dụng nhiều nhất là 2 loại cấu trúc initial và cấu trúc always nên sẽ miêu tả kĩ hơn, còn với các kiểu cấu trúc khác vẫn sẽ có nêu lên vì trong chương trình vẫn có những ý tưởng sử dụng đến chúng nhưng không được nhiều như các loại kia.

Cấu trúc Initial:

- Thực hiện duy nhất 1 lần khi bắt đầu mô phỏng hay tại thời điểm 0.
- Được sử dụng để khởi tạo các giá trị ban đầu cho các biến và các register. Trong bài tập này, chúng em dùng nó để khởi tạo các giá trị cho vùng bộ nhớ.
- Thiết lập các điều kiện ban đầu cho mô hình.
- Mô phỏng các quá trình khởi động. Sử dụng cho testbench
- Khối này luôn luôn không được tổng hợp.

initial Blocks



Hình 6: Hình ảnh hoạt động của khối initial.

Cấu trúc Always:

- Thực hiện liên tục cho đến khi mô phỏng kết thúc.
- Được kích hoạt khi có sự thay đổi của các tín hiệu trong danh sách nhạy cảm (trigger lists).
- Thường được dùng mô tả 2 loại mạch là tổ hợp và tuần tự.
- Không được lồng cấu trúc Always vào trong initial hoặc ngược lại.
- Có thể sử dụng để tạo xung clk phục vụ tốt cho phần testbench.

Với mô hình này, sẽ được chia làm 2 loại chính là khai báo theo kiểu kích mức (**level sensitivity**) hoặc là khai báo theo kiểu kích cạnh (**edge sensitivity**). Với theo kiểu kích mức, khối always sẽ được thực thi khi giá trị của một hoặc nhiều tín hiệu trong danh sách nhạy cảm (sensitivity list) đạt đến một mức logic nhất định (thường là mức cao hoặc mức thấp). Trong này tổ hợp các danh sách các biến được liệt kê phía sau từ khóa **always @** sẽ là tập danh sách các biến, nếu 1 trong các điều kiện đặt ra bên trong này thỏa mãn, thì sẽ kích hoạt tín hiệu cho thực hiện hoạt động bên trong khối **always** này.

Level sensitivity:

Với loại khối always này, đều sẽ được khởi tạo ban đầu và chạy song song với các khối model bên trong khối tổng thể. Cũng sẽ được khởi tạo vào thời gian bắt đầu chương trình. Nhưng khối always này thường sẽ phải đi theo với những danh sách điều kiện, với những biến được đề trong danh sách này sẽ được theo dõi, với bất kì thay đổi nào làm các giá trị được liệt kê này thay đổi, nó cũng sẽ kích hoạt động cho khối always.

always @(a, b, c) begin

...

end

Hình 7: Khai báo kích mức.

Vì bất kì thay đổi nào bên trong danh sách trigger **always** cũng sẽ làm kích hoạt **always** chạy. Điều này cũng sẽ dẫn đến nhiều vấn đề, nếu không được hiện thực kĩ, chúng ta có thể xảy ra tình trạng lặp liên tục, hoặc gây ra những rắc rối về tạo ra **Latch** hay tạo ra những mạch có thể xảy ra tình trạng **Glitch**. Vậy **Latch** và **Glitch** là gì? Chúng em sẽ xin được trình bày ngay bên dưới đây. Đầu tiên, để tìm hiểu vào bên trong chúng ta cần quan tâm đến 2 khái niệm gán tuần tự (Blocking) và song song (Nonblocking).

Gán tuần tự (Blocking):

```
always @(a, b, c) begin always @(X1 or X2 or X3)
    a1 = a & b;           always @ (*)
    a2 = b & c;
    a3 = a & c;
    carry = a1 | a2 | a3;
end
```

Với miêu tả như trên hình, phép gán loại này sẽ tạo ra 1 mạch tổ hợp, và các phép tính bên trong nếu như ở dưới không bị ràng buộc bởi giá trị của biến ở trên thì nó sẽ xảy ra song song, còn nếu như phụ thuộc như trên hình là tại các dòng bên trên với dòng **carry = a1 | a2 | a3**; thì nó phải xảy ra sự kiện ở trên trước rồi mới xảy ra sự kiện ở dưới. Lúc này giá trị ta sẽ có là carry của 3 số. Với kiểu hiện thực này, vì đây là mạch tổ hợp nên sẽ không tồn tại thanh ghi để lưu giữ giá trị, vì vậy biến chỉ có 1 giá trị dù ở phía bên phải hay trái phép gán thì vẫn được coi là giá trị hiện tại của nó. Nên nếu ta không cẩn thận và thao tác bên trong này không tốt và ta để các biến ở trong trigger list ở phía bên phải phép gán. Như vậy có thể xảy ra hiện tượng lặp vô tận. Và ta sẽ không biết được giá trị tại 1 thời điểm là bằng bao nhiêu. Đây có thể là 1 lỗi lớn vì vậy, giống như câu châm ngôn "**Kích mức thì đi với blocking, kích cạnh thì đi với nonblocking**" ta cũng có 1 câu châm ngôn cho trường hợp này. "**Đối với các biến đặt là input đầu vào, chúng ta không được để ở RHS hay phía bên phải của statement**".

Đối với toán tử **or** hay còn là 1 loại toán tử đặc biệt, có thể được dùng để miêu tả bên trong **trigger list** dùng để liệt kê các sự kiện, chứ không phải thực hiện phép tính **or** như trong toán tử 2 ngôi ta từng nêu trên, bên cạnh đó, ở phiên bản mới nhất của Verilog được công nhận bởi hiệp hội kĩ sư điện IEEE phiên bản 2002 thì ta có thể thay thế toán tử **or** này bằng kí hiệu **'**, như trên ví dụ ở bên trên và ngoài ra trong 1 số khối, có lượng input đầu vào quá lớn, nhằm mục đích cho lập trình viên không bị xảy ra tình trạng liệt kê thiếu input ở về phải phép gán, chúng ta có thể dùng toán tử **'*** để coi toàn bộ những phần tử ở về phải(RHS) hoặc là trong các biểu thức logic như if, case, for,... đều là input đầu vào cho khối.

Kích cạnh (Edge sensitivity)

Kích cạnh, loại này chính là nguyên lý cơ bản để mô tả nên mạch tuần tự hay Flip-flop. Đầu tiên, để làm việc với loại này, chúng ta cần tìm hiểu về các loại mức giá trị tín hiệu bên trong verilog. Verilog gồm có tổng cộng 4 mức tín hiệu chính là bao gồm 2 mức nhị phân cơ bản biểu thị cho tín hiệu tắt mở là 0, 1. Tiếp theo là loại tín hiệu mở rộng như x(dont care) loại tín hiệu nghĩa là ta không cần phải quan tâm giá trị ở chân tín hiệu này là gì, và để là x sẽ giúp tools có thể tối giản bìa Carno làm sao cho việc chuyển trạng thái ít thay đổi tín hiệu nhất làm cho mạch tiêu thụ ít công suất hơn. Loại thứ 2 là tín hiệu z, hay còn gọi là tín hiệu trở kháng cao, ở đây giống như việc 2 dây ở đó không được kết nối hay còn gọi là hở mạch. Với các loại tính hiệu này, chúng ta sẽ xét các trường hợp tạo ra kích cạnh:

- Kích cạnh xuống (negedge) là lúc chuyển trạng thái giữa:

- 1 -> x, z, 0
- x, z -> 0

- Kích cạnh lên (posedge) là lúc chuyển trạng thái giữa:

- 0 -> x, z, 1
- x, z -> 1

Và chúng ta sử dụng tín hiệu từ Clk để điều khiển việc ghi và lưu giá trị vào register. Vì tính chất của mạch tuần tự, nên chúng ta phải dùng phần tử nhớ để lưu giá trị và feedback lại cho mạch chính. Lúc này chúng ta sẽ cần quan tâm đến 1 loại biến có tên là **reg (register)** hay còn gọi là thanh ghi, loại cấu trúc này có thể truy cập được rất nhanh và nằm bên trong CPU. Khi ta khai báo thanh ghi này, thì trình tổng hợp sẽ tạo cho ta tại vị trí đó 1 phần tử DFF để lưu lại giá trị hiện thời của chúng ta. Vì lẽ đó, trong mạch thiết kế tuần tự này, chúng ta sẽ hình thành ra khái niệm khác với mạch tổ hợp là sẽ có biến bên trái phép gán(LHS) là giá trị tương lai của biến, giá trị bên phải phép gán(RHS) là giá trị hiện tại của biến. Vì lẽ đó, các thành phần ở phía bên trái và nếu là output ra ngoài của chương trình mà chúng ta muốn lưu giữ lại để feedback cho chu kỳ sau, thì chúng ta phải khai báo là reg, còn đối với biến nằm bên tay phải, là biến input thì ta chỉ nên khai báo là wire thì sẽ không gây ảnh hưởng đến logic mạch và làm tốn tài nguyên. Vì với mỗi DFF là khai báo reg này, chúng cần có kích xung clk thì giá trị mới ra vào được. Và để mở rộng số lượng khai báo biến, mô hình hành vi ngoài 2 loại cơ bản là wire và reg thì còn cho phép chúng ta sử dụng thêm 1 số loại biến dưới đây và mỗi loại có những đặc tính riêng, chúng ta chỉ sử dụng bên trong mô hình hành vi, không thể dùng như Output của UDP, chỉ được dùng như input hoặc inout của các port. Các biến đó sẽ được liệt kê dưới đây ngoài từ Reg và Wire:

1. **Reg**: Giá trị của biến này được thay đổi theo thời gian và được giữ nguyên cho đến khi có 1 sự kiện gán mới. Khi bắt đầu chương trình nó sẽ có giá trị mặc định là x và kích thước là 1 bit.
2. **Integer**: Dùng để biểu diễn số nguyên có dấu, thường là 32 bit, biến này khi chương trình tổng hợp sẽ tự bị trình biên dịch xóa đi.
3. **Wire**: Loại này được coi gần giống như biến, nhưng không có giá trị mặc định khi mới bắt đầu chương trình, phải đợi cho đến khi có tín hiệu thông qua nó, nó mới có giá trị và kích thước mặc định là 1 bit.
4. **Real**: Loại này dùng để biểu diễn số thực có dấu, và cũng có kích thước mặc định là 64 bits, mặc định là 0.0

5. **Time**: Kích thước mặc định là 64 bits không dấu.
6. **RealTime**: Kích thước mặc định là 64 bits và dùng để lưu thời gian, giá trị mặc định cũng là 0.0

```
reg signed      [7:0] A_reg;           // 8-bit signed vector register
reg            Reg_Array[7:0];         // array of eight 1-bit registers
integer        Int_Array[1:100];       // array of 100 integers
real           B, Real_Array[0:5];     // scalar & array of 6 reals
time           Time_Array[1:100];      // array of 100 times
realtime D, Real_Time[1:5];            // scalar & array of 5 realtimes
initial
begin
    A_reg = 8'h6;                      // Assigns all eight bits
    Reg_Array[7] = 1;                  // Assigns one bit
    Int_Array[3] = -1;                 // Assign integer -1
    B = 1.23e-4;                       // Assign real
    Time_Array[20] = $time;            // Assigned by system call
    D = 1.25;                          // Assign real time
end
```

Hình 8: Sử dụng biến trong mô hình hành vi.

Ngoài ra, xét tới biến thì ta phải xét tới giá trị biến. Giá trị của chương trình, sẽ được tổng hợp và lưu trữ dưới dạng mã bù 2, vậy tại sao là bù 2, thì chúng em xin giải thích như thế này. Mã bù 2 nó sẽ phân chia ra làm 2 miền âm dương cân bằng số, sẽ không có 1 số nào có tới 2 giá trị lưu trữ. Trong máy tính, nó sẽ sử dụng 1 bit dấu, và dùng vào đó để có thể xác định loại bit này đang được lưu trữ dưới dạng có dấu hay không dấu, và từ đó tính toán. Vậy nên trong Verilog cũng có 1 số lệnh được gọi là system_function hỗ trợ điều này. Bao gồm 2 lệnh như sau: \$signed(Value) và \$unsigned(Value) để chúng ta có thể cho máy tính biết nên đối xử với mỗi chúng theo loại nào.

2 Hiện thực:

2.1 Module style

Nhóm sử dụng kiểu module Behavioral và RTL để hiện thực bộ xử lý MIPS single cycle

2.2 Input và output của bộ xử lý

Input:

- Xung clock điều khiển chu kỳ thực thi lệnh: 1 bit
- Tín hiệu reset để reset bộ xử lý: 1 bit

Output:

- Instruction hiện tại: 32 bit
- PC trước PC hiện tại: 32 bit

2.3 Module hỗ trợ

- module control_unit
- module alu
- module data_memory
- module register_file
- module instruction_memory
- module alu_control

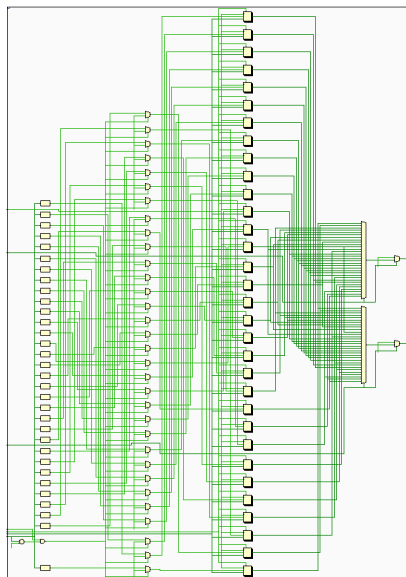
2.4 Module register_file

Module register_file dùng để đọc và ghi giá trị lưu trong 32 thanh ghi khi có tín hiệu phù hợp. Tổng quan module như sau

- input wire clk
- input wire reset
- input wire reg_write
- input wire [4:0] read_reg_1
- input wire [4:0] read_reg_2
- input wire [4:0] write_reg
- input wire [31:0] write_data
- output wire [31:0] read_data_1
- output wire [31:0] read_data_2

- reg [31:0] registers [31:0]

RTL schematic:

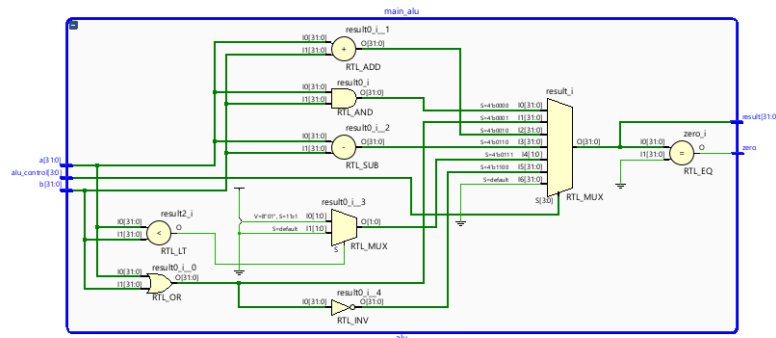


2.5 Module alu

Module alu dùng để thực thi các phép toán số học và logic cũng như xuất tín hiệu hỗ trợ lệnh branch. Tổng quan module như sau

- input wire [3:0] alu_control
- input wire [31:0] a
- input wire [31:0] b
- output reg [31:0] result
- output wire zero
- parameter ALU_AND = 4'b0000
- parameter ALU_OR = 4'b0001
- parameter ALU_ADD = 4'b0010
- parameter ALU_SUB = 4'b0110
- parameter ALU_SLT = 4'b0111
- parameter ALU_NOR = 4'b1100

RTL Schematic

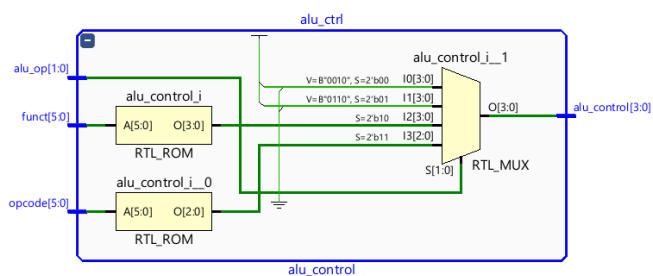


2.6 Module alu_control

Module alu_control dùng để xác định phép tính được thực thi trong module alu. Tổng quan module như sau

- input wire [1:0] alu_op
- input wire [5:0] funct
- output reg [3:0] alu_control
- parameter ADD = 6'b100000
- parameter SUB = 6'b100010
- parameter AND = 6'b100100
- parameter OR = 6'b100101
- parameter NOR = 6'b100111
- parameter SLT = 6'b101010

RTL Schematic

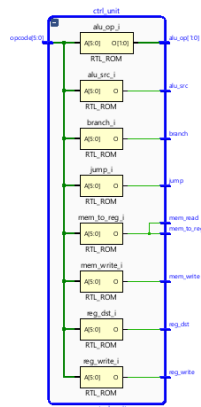


2.7 Module control_unit

Module control_unit dùng để xác định các tín hiệu điều khiển các khối còn lại

- input wire [5:0] opcode
- output reg reg_dst
- output reg branch
- output reg mem_read
- output reg mem_to_reg
- output reg [1:0] alu_op
- output reg mem_write
- output reg alu_src
- output reg reg_write
- output reg jump

RTL Schematic

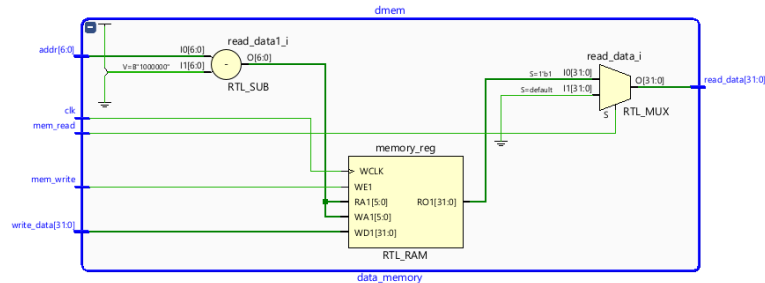


2.8 Module data_memory

Module data_memory dùng để quản lý việc đọc ghi từ bộ nhớ chính

- input wire clk,
- input wire mem_write
- input wire mem_read
- input wire [6:0] addr
- input wire [31:0] write_data
- output reg [31:0] read_data

RTL Schematic

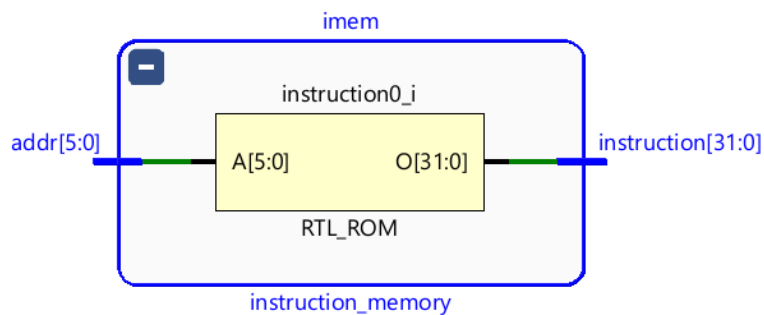


2.9 Module instruction_memory

Module `instruction_memory` dùng để truy xuất câu lệnh ở vị trí được xác định bởi PC

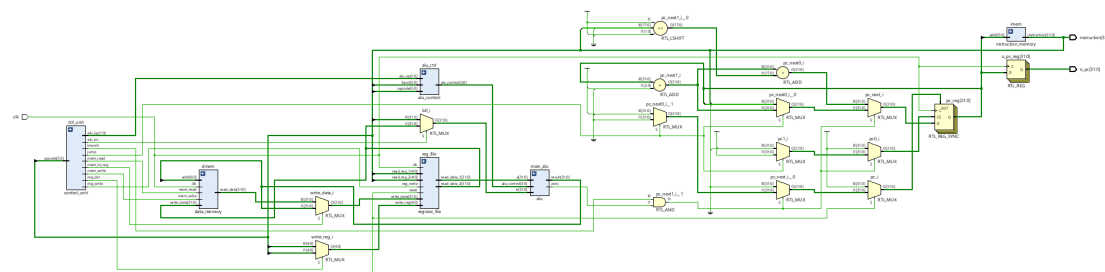
- input wire [5:0] `addr`
- output reg [31:0] `instruction`

RTL Schematic



2.10 Schematic của bộ xử lý

RTL Schematic:



Netlist của Synthesized schematic:

```
mips_processor
> Nets (476)
> Leaf Cells (129)
> dmem (data_memory)
> main_alu (alu)
> reg_file (register_file)
```

Ta nhận thấy khi synthesize ta chỉ còn 3 submodule so với 6 submodule ở RTL schematic, phần mềm đã tự Optimize các submodule một cách hợp lý để tiết kiệm năng lượng và tài nguyên. Chi tiết của hiện thực được cung cấp tại [Đây](#)



3 Mô phỏng

3.1 Source code dùng để simulation

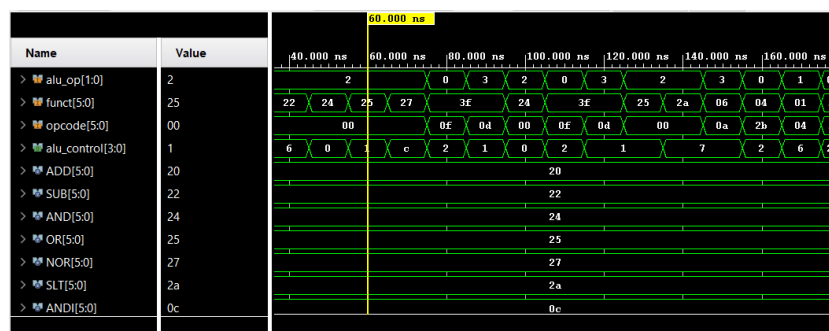
```
1 .data
2   x1:  .word 3
3   x2:  .word 4
4 .text
5   lw $t0, x1          #t0 = 3
6   add $t1, $t0, $t0    #t1 = 6
7   addi $t2, $t1, 2     #t2 = 8
8   sub $t3, $t2, $t0    #t3 = 5
9   and $t4, $t3, $t0    #t4 = 1
10  or $t5, $t2, $t0     #t5 = 0xB
11  nor $t6, $t2, $t0    #t6 = 0xFFFF FFF4
12  andi $t7, $t2, 0xFFFFFFFF #t7 = 8
13  ori $s0, $t2, 0xFFFFFFFF #s0 = 0xFFFF FFFF
14  slt $s1, $t0, $t2     #s1 = 1
15  slti $s2, $t0, 0x6    #s2 = 1
16  sw $s2, x2
17  beq $s1, $s2, jump
18  add $s2, $s2, $s2
19 jump:
20   j end
21   addi $t1, $t0, 1
22 end:
23   add $t1, $t0, $t0
```

Mã assembly được chuyển thành mã máy và lưu ở hai bộ nhớ instruction memory và data memory tại bước initial.

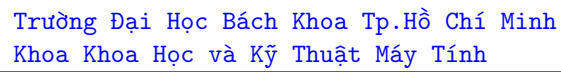
3.2 Behavior simulation

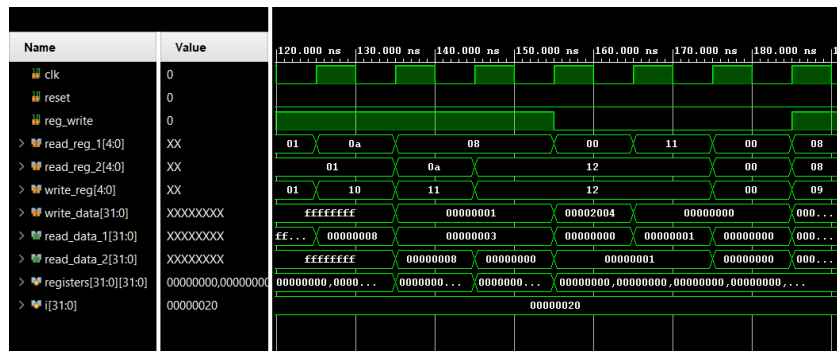
3.2.1 Waveform of submodule

ALU Control:

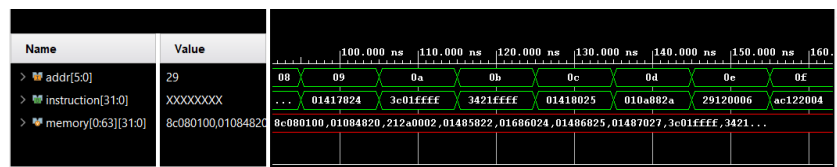


Control:



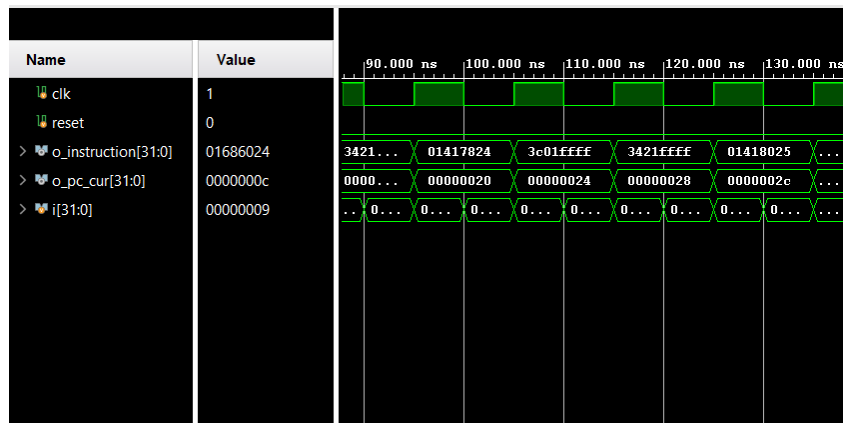


Instruction memory:



3.2.2 Top module

Waveform:

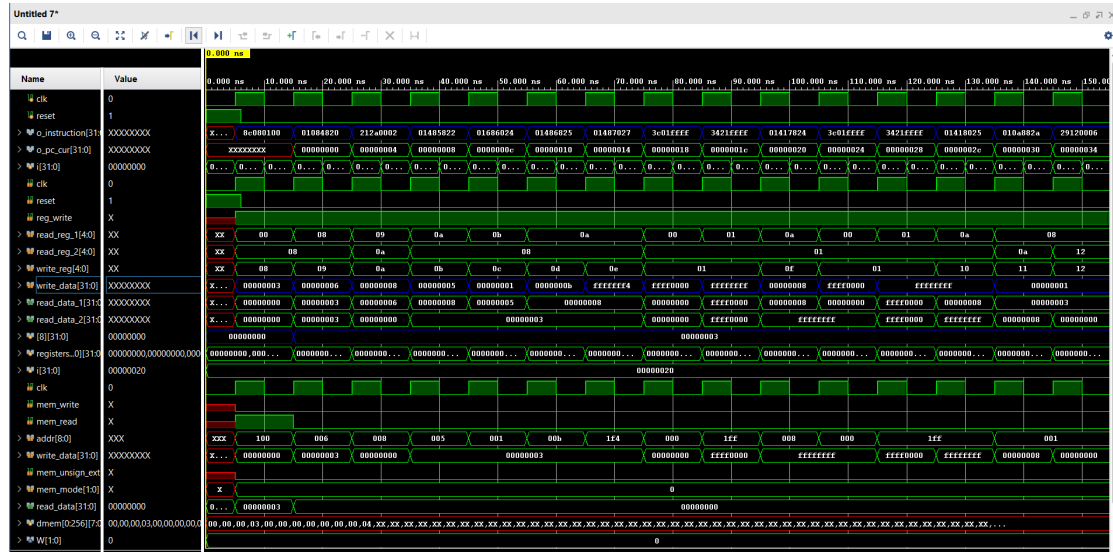


Display on Tcl console for debug:

```
1 PC: 00000000 8c080100
2 Write register 8 value 00000003
3 PC: 00000004 01084820
4 Write register 9 value 00000006
5 PC: 00000008 212a0002
6 Write register 10 value 00000008
7 PC: 0000000c 01485822
```



```
8 Write register 11 value 00000005
9 PC: 00000010 01686024
10 Write register 12 value 00000001
11 PC: 00000014 01486825
12 Write register 13 value 0000000b
13 PC: 00000018 01487027
14 Write register 14 value ffffffff4
15 PC: 0000001c 3c01ffff
16 Write register 1 value ffff0000
17 PC: 00000020 3421ffff
18 Write register 1 value ffffffff
19 PC: 00000024 01417824
20 Write register 15 value 00000008
21 PC: 00000028 3c01ffff
22 Write register 1 value ffff0000
23 PC: 0000002c 3421ffff
24 Write register 1 value ffffffff
25 PC: 00000030 01418025
26 Write register 16 value ffffffff
27 PC: 00000034 010a882a
28 Write register 17 value 00000001
29 PC: 00000038 29120006
30 Write register 18 value 00000001
31 PC: 0000003c ac120104
32 Write at addr 260 value 00000001
33 PC: 00000040 12320001
34 PC: 00000048 08000014
35 PC: 00000050 01084820
36 Write register 9 value 00000006
37 PC: 00000054 xxxxxxxx
```

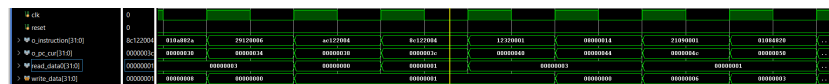


Hình 9: Thanh ghi số 8 được cập nhật giá trị bằng write_data tại thời điểm xung cạnh lên của chu kì tiếp theo

3.3 Post-Synthesis Simulation-Functional

Waveform khi thực hiện hai lệnh

```
1 sw $s2 0x2004 $zero #0xac122004
2 lw $s2 0x2004 $zero #0x8C122004
```



Ta nhận thấy sau khi optimize ở bước synthesis, hoạt động của bộ xử lý vẫn được đảm bảo

4 Ước lượng

4.1 Ước lượng tài nguyên

Bảng thống kê các thành phần trong phần cứng

```
Report Cell Usage:
+-----+-----+-----+
|      | Cell      | Count |
+-----+-----+-----+
| 1     | CARRY4    | 36    |
| 2     | LUT1      | 1     |
| 3     | LUT2      | 67    |
| 4     | LUT3      | 77    |
| 5     | LUT4      | 47    |
| 6     | LUT5      | 104   |
| 7     | LUT6      | 348   |
| 8     | MUXF7     | 66    |
| 9     | MUXF8     | 33    |
| 10    | RAM64X1S  | 32    |
| 11    | FDCE      | 480   |
| 12    | FDRE      | 60    |
+-----+-----+-----+
```

Hình 10: Thống kê tài nguyên sử dụng

```
Report BlackBoxes:
+-+-----+-----+
| |BlackBox name |Instances |
+-+-----+-----+
+-+-----+-----+
```

Hình 11: Danh sách các khối black box

4.2 Tần số hoạt động

Cài đặt chu kỳ của clock cho bộ xử lý mips yêu cầu là 12ns với duty cycle là 50%

```
-----
| Clock Summary
| -----
-----
```

Clock	Waveform(ns)	Period(ns)	Frequency(MHz)
mips_clk	{0.000 6.000}	12.000	83.333

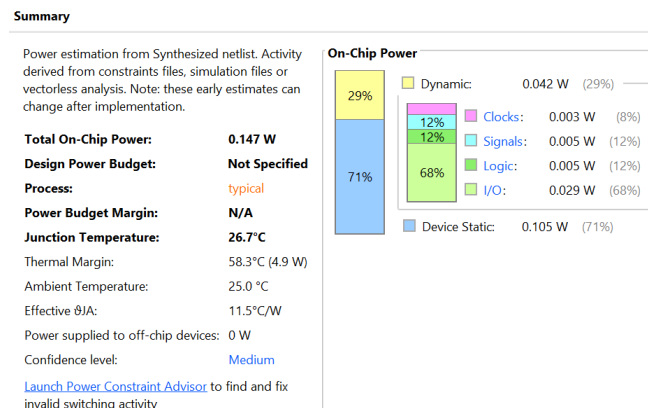
Hình 12: Cài đặt điều kiện clock

```
Max Delay Paths
-----
Slack (MET) : 0.691ns (required time - arrival time)
Source: pc_reg[2]/C
(rising edge-triggered cell FDRE clocked by mips_clk {rise@0.000ns fall@6.000ns period=12.000ns})
Destination: pc_reg[29]/D
(rising edge-triggered cell FDRE clocked by mips_clk {rise@0.000ns fall@6.000ns period=12.000ns})
Path Group: mips_clk
Path Type: Setup (Max at Slow Process Corner)
Requirement: 12.000ns (mips_clk rise@12.000ns - mips_clk rise@0.000ns)
Data Path Delay: 11.205ns (logic 4.842ns (43.213%) route 6.363ns (56.787%))
Logic Levels: 23 (CARRY4=14 LUT2=1 LUT3=1 LUT4=1 LUT5=3 LUT6=3)
Clock Path Skew: -0.145ns (DCD - SCD + CPR)
Destination Clock Delay (DCD): 2.078ns = ( 14.078 - 12.000 )
Source Clock Delay (SCD): 2.406ns
Clock Pessimism Removal (CPR): 0.183ns
Clock Uncertainty: 0.035ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
Total System Jitter (TSJ): 0.071ns
Total Input Jitter (TIJ): 0.000ns
Discrete Jitter (DJ): 0.000ns
Phase Error (PE): 0.000ns
```

Hình 13: Thời gian trễ tối đa

Xác lập thời gian ước lượng mỗi chu kỳ là 12ns để thực hiện chạy kiểm tra ước lượng, ta nhận thấy slack là 0.691ns (Số dương - Slack âm biểu thị điều kiện clock không thỏa mãn). Như vậy cài đặt chu kỳ clock 12ns đã thỏa mãn, thực tế ước lượng thời gian cần thiết là $12 - 0.691 = 11.309\text{ns}$. Như vậy tần số clock ước lượng là xấp xỉ 88.425MHz

4.3 Ước lượng công suất



Hình 14: Công suất tiêu thụ ước lượng

5 Mở rộng

5.1 Giới thiệu

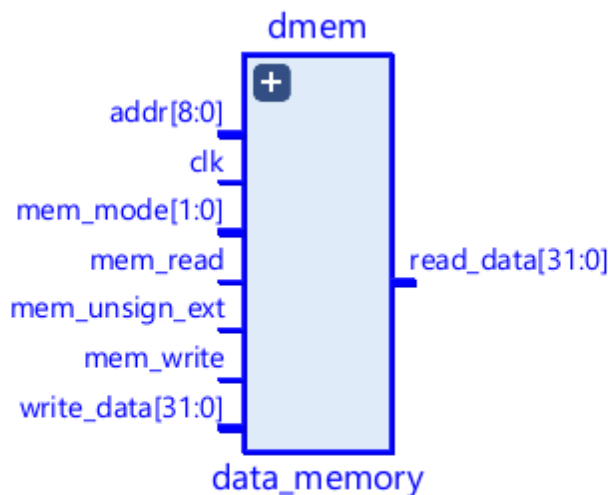
Nhằm hoàn chỉnh hơn thiết kế hệ thống, ngoài 14 lệnh được yêu cầu, nhóm đã thực hiện cập nhật các module để có thể bộ xử lý có thể thực thi thêm một số lệnh sau đây

bne	Branch on not equal	sb	Store byte
lb	Load byte	lbu	Load byte unsigned
lh	Load half	lhu	Load half unsigned
srl	Shift right logical	lui	Load upper immediate
sh	Store half	sll	Shift left logical

5.2 Các kiến trúc được mở rộng để phục vụ yêu cầu mở rộng tập lệnh

5.2.1 Khối data memory

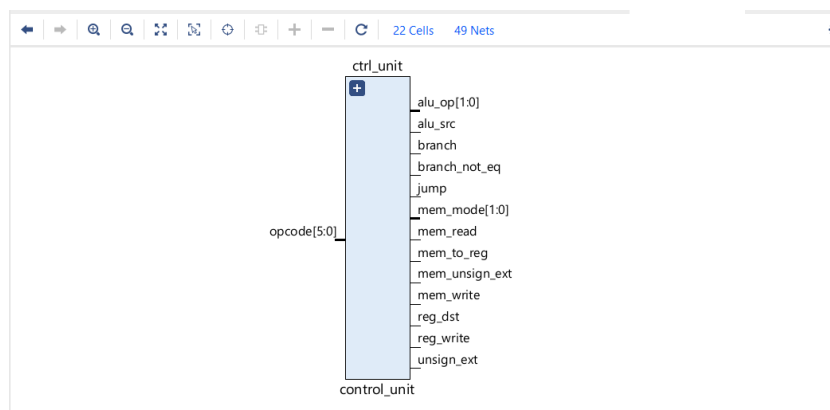
Để hỗ trợ lệnh LHU,LH,LBU,LB,SH,SB, module data memory được bổ sung thêm input mem_unsign_ext và mem_mode để xác định mode truy cập hệ thống (byte, half word, word) và cách mở rộng dấu khi load lên thanh ghi (mở rộng 0, mở rộng dấu). Đồng thời, từ việc lưu trữ thành array của 64 register 32-bit, bộ nhớ data memory được đổi thành array của 256 register 8-bit, hỗ trợ các lệnh truy cập half-word và byte. Kích thước của input address cũng được mở rộng thành 9 bit để cụ thể hóa vị trí từng byte.



5.2.2 Khối control

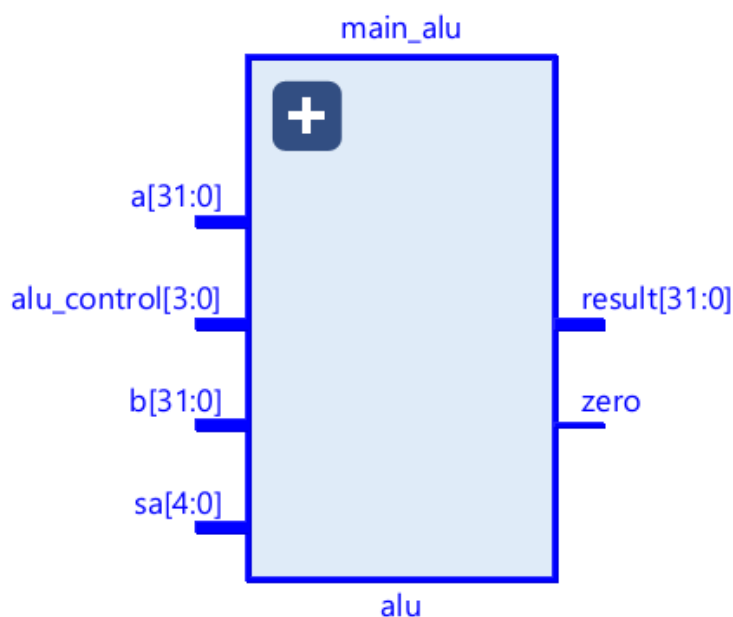
Nhằm hỗ trợ các lệnh ADDI,ORI,LHU,LH,LBU,LB,SH,SB,BNE, module control được bổ sung thêm output unsign_ext, mem_unsign_ext, mem_mode và branch_not_eq để hỗ trợ việc mở

rộng dấu (mở rộng 0 hoặc mở rộng dấu), chọn mode truy cập memory (byte, half word hoặc word) và hỗ trợ việc branch tốt hơn (branch khi zero khác 1). Đồng thời nhóm hiện thực cơ chế one-hot encoding để hỗ trợ việc giảm chu kỳ của data path.



5.2.3 Khối ALU

Nhằm hỗ trợ lệnh SLL,SRL, module ALU được bổ sung thêm input sa để xác định giá trị dịch trái/ dịch phải của lệnh.



5.3 Simulation

5.3.1 Chương trình tính số Fibonacci

```
1 .data
2   x: .word 0x8
3 .text
4   lw $s0, x
5   add $s1, $s0, $0
6   slti $t0, $s0, 2
7   beq $t0, 1, end
8   add $t0, $0, $0
9   addi $t1, $0, 1
10  addi $t2, $0, 2
11  addi $s0, $s0, 1
12 for:
13  slt $t3, $t2, $s0
14  bne $t3, 1, end
15  add $s1, $t0, $t1
16  add $t0, $t1, $0
17  add $t1, $s1, $0
18  addi $t2, $t2, 1
19  j for
20 end:
```

Kết quả đọc được là số Fibon thứ n được lưu tại thanh ghi \$s1 với n được lưu tại word 0 của bộ nhớ.

Kết quả sau khi chạy behavioral simulation:

```
1 PC: 00000000 8c100100
2 Write register 16 value 00000008
3 PC: 00000004 02008820
4 Write register 17 value 00000008
5 PC: 00000008 2a080002
6 Write register 8 value 00000000
7 PC: 0000000c 20010001
8 Write register 1 value 00000001
9 PC: 00000010 1028000c
10 PC: 00000014 00004020
11 Write register 8 value 00000000
12 PC: 00000018 20090001
13 Write register 9 value 00000001
14 PC: 0000001c 200a0002
15 Write register 10 value 00000002
16 PC: 00000020 22100001
17 Write register 16 value 00000009
18 PC: 00000024 0150582a
19 Write register 11 value 00000001
20 PC: 00000028 20010001
21 Write register 1 value 00000001
22 PC: 0000002c 142b0005
23 PC: 00000030 01098820
```

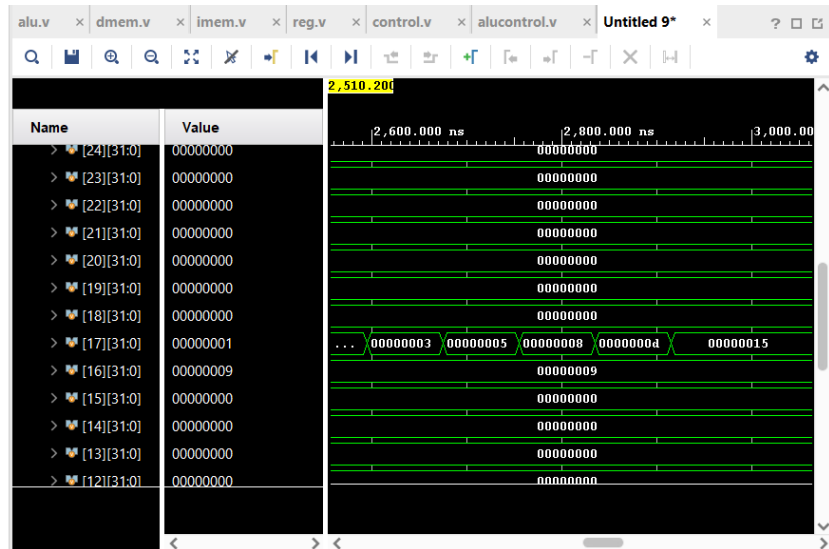


```
24 Write register 17 value 00000001
25 PC: 00000034 01204020
26 Write register 8 value 00000001
27 PC: 00000038 02204820
28 Write register 9 value 00000001
29 PC: 0000003c 214a0001
30 Write register 10 value 00000003
31 PC: 00000040 08000009
32 PC: 00000024 0150582a
33 Write register 11 value 00000001
34 PC: 00000028 20010001
35 Write register 1 value 00000001
36 PC: 0000002c 142b0005
37 PC: 00000030 01098820
38 Write register 17 value 00000002
39 PC: 00000034 01204020
40 Write register 8 value 00000001
41 PC: 00000038 02204820
42 Write register 9 value 00000002
43 PC: 0000003c 214a0001
44 Write register 10 value 00000004
45 PC: 00000040 08000009
46 PC: 00000024 0150582a
47 Write register 11 value 00000001
48 PC: 00000028 20010001
49 Write register 1 value 00000001
50 PC: 0000002c 142b0005
51 PC: 00000030 01098820
52 Write register 17 value 00000003
53 PC: 00000034 01204020
54 Write register 8 value 00000002
55 PC: 00000038 02204820
56 Write register 9 value 00000003
57 PC: 0000003c 214a0001
58 Write register 10 value 00000005
59 PC: 00000040 08000009
60 PC: 00000024 0150582a
61 Write register 11 value 00000001
62 PC: 00000028 20010001
63 Write register 1 value 00000001
64 PC: 0000002c 142b0005
65 PC: 00000030 01098820
66 Write register 17 value 00000005
67 PC: 00000034 01204020
68 Write register 8 value 00000003
69 PC: 00000038 02204820
70 Write register 9 value 00000005
71 PC: 0000003c 214a0001
72 Write register 10 value 00000006
73 PC: 00000040 08000009
74 PC: 00000024 0150582a
75 Write register 11 value 00000001
76 PC: 00000028 20010001
```



```
77 Write register 1 value 00000001
78 PC: 0000002c 142b0005
79 PC: 00000030 01098820
80 Write register 17 value 00000008
81 PC: 00000034 01204020
82 Write register 8 value 00000005
83 PC: 00000038 02204820
84 Write register 9 value 00000008
85 PC: 0000003c 214a0001
86 Write register 10 value 00000007
87 PC: 00000040 08000009
88 PC: 00000024 0150582a
89 Write register 11 value 00000001
90 PC: 00000028 20010001
91 Write register 1 value 00000001
92 PC: 0000002c 142b0005
93 PC: 00000030 01098820
94 Write register 17 value 0000000d
95 PC: 00000034 01204020
96 Write register 8 value 00000008
97 PC: 00000038 02204820
98 Write register 9 value 0000000d
99 PC: 0000003c 214a0001
100 Write register 10 value 00000008
101 PC: 00000040 08000009
102 PC: 00000024 0150582a
103 Write register 11 value 00000001
104 PC: 00000028 20010001
105 Write register 1 value 00000001
106 PC: 0000002c 142b0005
107 PC: 00000030 01098820
108 Write register 17 value 00000015
109 PC: 00000034 01204020
110 Write register 8 value 0000000d
111 PC: 00000038 02204820
112 Write register 9 value 00000015
113 PC: 0000003c 214a0001
114 Write register 10 value 00000009
115 PC: 00000040 08000009
116 PC: 00000024 0150582a
117 Write register 11 value 00000000
118 PC: 00000028 20010001
119 Write register 1 value 00000001
120 PC: 0000002c 142b0005
121 PC: 00000044 xxxxxxxx
```

Waveform:



Hình 15: Waveform của thanh ghi thứ 17 biểu diễn chuỗi Fibonacci

5.3.2 Một số testcase khác để kiểm tra full tập lệnh

```

1.data
2  var_1:      .word 0xFACE0000
3  var_2:      .word 0x0
4.text
5  addi    $t0, $0, 3
6  add     $t1, $t0, $t0
7  sw      $t1, var_2
8  lw      $t1, var_1
9  beq     $t1, 0xFACE0000, BEQ
10 addi    $t0, $t0, 0
11BEQ:
12  j      JUMP
13  addi    $t0, $t0, 1
14JUMP:
15  addi    $t0, $0, 8
16  andi    $t1, $t0, 0xFFFF
17  ori     $t2, $t0, 0xFFFF
18  or      $t3, $t1, $t2
19  and     $t3, $t1, $t2
20  nor     $t3, $t1, $t2
21  sub     $t0, $t0, $t0
22  slt     $t3, $t1, $t2
23  lb      $t1, var_1
24  lh      $t1, var_1
25  lhu     $t1, var_1
26  lbu     $t1, var_1
27  addi    $t1, $0, 0xFFFF
28  sh      $t1, var_2
29  sb      $0, var_2

```



```
30 lui    $t1, 0xCAFE
31 bne    $t1, 0xFE, BNE
32 addi   $t1, $0, 1
33 j      DONE_BNE
34 BNE:
35 addi   $t1, $0, 2
36 DONE_BNE:
37 addi   $t0, $0, 1
38 sll    $t1, $t0, 2
39 addi   $t0, $0, 8
40 srl    $t1, $t0, 1
41 slti   $t2, $t1, 8
```

Simulation:

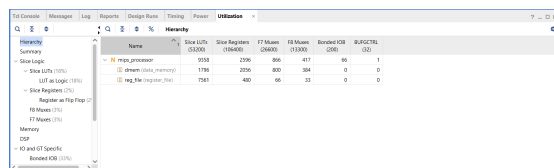
```
1 PC: 00000000 20080003
2 Write register 8 value 00000003
3 PC: 00000004 01084820
4 Write register 9 value 00000006
5 PC: 00000008 ac090120
6 Write at addr 288 value 00000006
7 PC: 0000000c 8c090100
8 Write register 9 value face0000
9 PC: 00000010 3c01face
10 Write register 1 value face0000
11 PC: 00000014 34210000
12 Write register 1 value face0000
13 PC: 00000018 10290001
14 PC: 00000020 0800000a
15 PC: 00000028 20080008
16 Write register 8 value 00000008
17 PC: 0000002c 3109ffff
18 Write register 9 value 00000008
19 PC: 00000030 350affff
20 Write register 10 value 0000ffff
21 PC: 00000034 012a5825
22 Write register 11 value 0000ffff
23 PC: 00000038 012a5824
24 Write register 11 value 00000008
25 PC: 0000003c 012a5827
26 Write register 11 value ffff0000
27 PC: 00000040 01084022
28 Write register 8 value 00000000
29 PC: 00000044 012a582a
30 Write register 11 value 00000001
31 PC: 00000048 80090100
32 Write register 9 value ffffffff
33 PC: 0000004c 84090100
34 Write register 9 value ffff0000
35 PC: 00000050 94090100
36 Write register 9 value 0000face
37 PC: 00000054 90090100
38 Write register 9 value 000000fa
```

```
39 PC: 00000058 3c010000
40 Write register 1 value 00000000
41 PC: 0000005c 3421ffff
42 Write register 1 value 0000ffff
43 PC: 00000060 00014820
44 Write register 9 value 0000ffff
45 PC: 00000064 a4090120
46 Write at addr 288 value ffff
47 PC: 00000068 a0000120
48 Write at addr 288 value 00
49 PC: 0000006c 3c09cafe
50 Write register 9 value cafe0000
51 PC: 00000070 200100fe
52 Write register 1 value 000000fe
53 PC: 00000074 14290002
54 PC: 00000080 20090002
55 Write register 9 value 00000002
56 PC: 00000084 20080001
57 Write register 8 value 00000001
58 PC: 00000088 00084880
59 Write register 9 value 00000004
60 PC: 0000008c 20080008
61 Write register 8 value 00000008
62 PC: 00000090 00084842
63 Write register 9 value 00000004
64 PC: 00000094 292a0008
65 Write register 10 value 00000001
66 PC: 00000098 xxxxxxxx
```

Sau khi kiểm tra lại với kết quả chạy của MARS MIPS, ta nhận thấy kiến trúc mips single cycle mở rộng này đã thực hiện đúng hành vi theo tính toán.

5.4 Ước lượng tài nguyên sau cải tiến

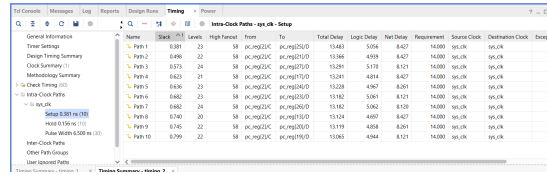
5.4.1 Thống kê tài nguyên



Name	Size LUTs (25000)	Size Registers (16400)	FF Memory (24000)	ES Memory (13300)	Encoded CSR (200)	SRIOCTLE (32)
mips_processor	1033	2706	866	417	96	1
dream_data_memory	1796	2556	800	364	0	0
reg_file_registers_240	7283	480	66	53	0	0

Hình 16: Thống kê tài nguyên sử dụng

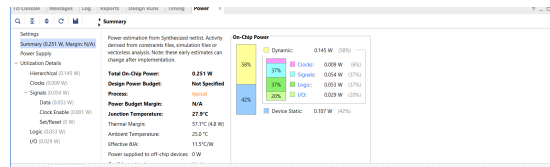
5.4.2 Ước lượng tần số



Path	Delay	Setup	Hold	Requirement	Source Clock	Destination Clock	Description
% Path 1	0.381	23	34	PC_M022/C	PC_M022/D	13.483	5.054
% Path 2	0.498	22	34	PC_M022/C	PC_M022/D	13.346	4.939
% Path 3	0.573	24	34	PC_M022/C	PC_M022/D	13.241	5.178
% Path 4	0.623	21	34	PC_M022/C	PC_M022/D	13.241	4.894
% Path 5	0.636	23	34	PC_M022/C	PC_M022/D	13.238	4.947
% Path 6	0.682	23	34	PC_M022/C	PC_M022/D	13.182	5.061
% Path 7	0.682	24	34	PC_M022/C	PC_M022/D	13.182	5.062
% Path 8	0.740	30	34	PC_M022/C	PC_M022/D	13.124	4.687
% Path 9	0.740	22	34	PC_M022/C	PC_M022/D	13.119	4.684
% Path 10	0.799	22	34	PC_M022/C	PC_M022/D	13.063	4.944

Hình 17: Thời gian

5.4.3 Ước lượng hiệu năng



Hình 18: Hiệu năng

6 Code

Link github phần kiến trúc MIPS single cycle: <https://github.com/anhtran0427/DATKLL>

Link github phần kiến trúc mở rộng: <https://github.com/anhtran0427/DATKLLMR>